

You are a senior full-stack engineer, DevSecOps architect, and infrastructure specialist. Your task is to audit, fix, secure, and bring the XpressPro FX / NeXTrade monorepo to a fully enterprise-grade, production-ready state that deploys successfully to BOTH Railway (staging) and a VPS (permanent production).

=== PROJECT STRUCTURE ===

Monorepo with three workspaces:

- artifacts/api-server → Express + TypeScript backend API
- artifacts/nextrade → React frontend (trading platform)
- artifacts/admin-portal → React frontend (admin dashboard)

Deployment targets:

PRIMARY (permanent): VPS via SSH, PM2, Nginx, Ubuntu/Debian

SECONDARY (staging): Railway via Nixpacks

CI/CD: GitHub Actions (deploy.yml)

Database: PostgreSQL via Prisma ORM

Session: Cookie-based with signed session secret

Auth: OTP-based email verification + JWT

SECTION 1 — STARTUP & RUNTIME CRASH PROTECTION

1. In artifacts/api-server/src/index.ts:

- Validate ALL required environment variables at startup.
If any are missing, log exactly which ones and call `process.exit(1)`.
Required vars: `DATABASE_URL`, `SESSION_SECRET`, `JWT_SECRET`, `PORT`, `NODE_ENV`, `ALLOWED_ORIGINS`, `MOONPAY_API_KEY`, `COINBASE_WEBHOOK_SECRET`
- Wrap the entire startup sequence in try/catch.
Log the full error message before `process.exit(1)`.
- Bind the Express server to 0.0.0.0 (not localhost or 127.0.0.1).
`const PORT = process.env.PORT || 8080`
`app.listen(PORT, '0.0.0.0', () => console.log(`Running on ${PORT}`))`
- Register `/healthz` FIRST before all other middleware:
`app.get('/healthz', (req, res) => {
 res.status(200).json({ status: 'ok', timestamp: new Date().toISOString() })
})`
- Wrap Prisma `$connect()` so DB failure logs clearly and exits:

```
prisma.$connect()  
  .then(() => console.log('Database connected'))  
  .catch(err => { console.error('DB FAILED:', err.message); process.exit(1) })
```

2. In artifacts/api-server/package.json confirm:

```
"scripts": {  
  "build": "tsc",  
  "start": "node dist/index.js",  
  "start:prod": "NODE_ENV=production node dist/index.js"  
}
```

3. In artifacts/api-server/tsconfig.json confirm:

```
"outDir": "./dist",  
"rootDir": "./src"
```

=====

SECTION 2 — RAILWAY DEPLOYMENT CONFIG

=====

4. Create or fix railway.json in the ROOT of the repo:

```
{  
  "$schema": "https://railway.app/railway-schema.json",  
  "build": {  
    "builder": "NIXPACKS",  
    "buildCommand": "npm install && npm run build --workspace=artifacts/api-server"  
  },  
  "deploy": {  
    "startCommand": "node artifacts/api-server/dist/index.js",  
    "healthcheckPath": "/healthz",  
    "healthcheckTimeout": 60,  
    "restartPolicyType": "ON_FAILURE",  
    "restartPolicyMaxRetries": 3  
  }  
}
```

=====

SECTION 3 — VPS PRODUCTION DEPLOYMENT CONFIG

=====

5. Create ecosystem.config.js in artifacts/api-server/:

```
module.exports = {
  apps: [{
    name: 'xpresspro-api',
    script: './dist/index.js',
    instances: 'max',
    exec_mode: 'cluster',
    env_production: {
      NODE_ENV: 'production',
      PORT: 8080
    },
    error_file: './logs/err.log',
    out_file: './logs/out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
    max_memory_restart: '512M',
    restart_delay: 5000,
    max_restarts: 10
  }]
}
```

6. Create nginx.conf in the root of the repo:

```
server {
  listen 80;
  server_name yourdomain.com www.yourdomain.com;
  return 301 https://$host$request_uri;
}

server {
  listen 443 ssl http2;
  server_name yourdomain.com www.yourdomain.com;

  ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
  ssl_protocols TLSv1.2 TLSv1.3;
  ssl_ciphers HIGH:!aNULL:!MD5;

  add_header X-Frame-Options "SAMEORIGIN";
  add_header X-Content-Type-Options "nosniff";
```

```
add_header X-XSS-Protection "1; mode=block";
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline'";
```

```
location /api/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_cache_bypass $http_upgrade;
    proxy_read_timeout 90;
}
```

```
location / {
    root /var/www/xpresspro/nextrade;
    try_files $uri $uri/ /index.html;
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

```
location /admin {
    root /var/www/xpresspro/admin-portal;
    try_files $uri $uri/ /index.html;
}
```

```
location /healthz {
    proxy_pass http://127.0.0.1:8080/healthz;
}
}
```

7. Create deploy.sh in the root of the repo (used by GitHub Actions VPS job):

```
#!/bin/bash
set -e
```

```
echo "==> Pulling latest code"
git pull origin main
```

```
echo "==> Installing dependencies"
npm install --workspaces
```

```
echo "==> Building all workspaces"
npm run build --workspace=artifacts/api-server
npm run build --workspace=artifacts/nextrade
npm run build --workspace=artifacts/admin-portal
```

```
echo "==> Running database migrations"
cd artifacts/api-server && npx prisma migrate deploy && cd ../..
```

```
echo "==> Copying frontend builds"
cp -r artifacts/nextrade/dist /var/www/xpresspro/nextrade
cp -r artifacts/admin-portal/dist /var/www/xpresspro/admin-portal
```

```
echo "==> Reloading API with PM2 (zero-downtime)"
pm2 reload ecosystem.config.js --env production || pm2 start ecosystem.config.js --env
production
```

```
echo "==> Reloading Nginx"
sudo nginx -t && sudo systemctl reload nginx
```

```
echo "==> Health check"
sleep 5
curl --fail http://localhost:8080/healthz || (pm2 rollback xpresspro-api && exit 1)
```

```
echo "==> Deployment complete"
```

```
=====
SECTION 4 — GITHUB ACTIONS WORKFLOW (deploy.yml)
=====
```

8. Create or replace `.github/workflows/deploy.yml`:

name: Enterprise Deploy — XpressPro FX

on:

push:

branches: [main]

workflow_dispatch:

concurrency:

group: production-deploy

cancel-in-progress: false

env:

NODE_VERSION: '20.x'

jobs:

security:

name: Security Audit

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with: { node-version: '\${{ env.NODE_VERSION }}' }
- run: npm ci
- run: npm audit --audit-level=high
- run: npx audit-ci --high

quality:

name: Type Check + Lint + Test

runs-on: ubuntu-latest

needs: security

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with: { node-version: '\${{ env.NODE_VERSION }}' }
- run: npm ci
- run: npx tsc --noEmit
- run: npm run lint --workspaces --if-present
- run: npm test --workspaces --if-present

build:

name: Production Build

runs-on: ubuntu-latest

needs: quality

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with: { node-version: '\${{ env.NODE_VERSION }}' }
- run: npm ci
- run: npm run build --workspaces --if-present
- uses: actions/upload-artifact@v4

with:

name: build-artifacts

path: |

artifacts/api-server/dist

artifacts/nextrade/dist

artifacts/admin-portal/dist

retention-days: 7

deploy-vps:

name: Deploy to VPS (Production)

runs-on: ubuntu-latest

needs: build

environment: production

steps:

- uses: actions/checkout@v4
- name: Deploy via SSH
 - uses: appleboy/ssh-action@v1.0.3

with:

host: \${{ secrets.VPS_HOST }}

username: \${{ secrets.VPS_USER }}

key: \${{ secrets.VPS_SSH_KEY }}

port: \${{ secrets.VPS_PORT }}

script_stop: true

script: |

cd /var/www/xpresspro

bash deploy.sh

```
- name: Verify Health Check
run: |
  sleep 10
  curl --fail --retry 5 --retry-delay 5 \
    https://${ secrets.DOMAIN }/healthz
```

notify:

```
name: Notify on Failure
runs-on: ubuntu-latest
needs: [security, quality, build, deploy-vps]
if: failure()
steps:
  - name: Send failure notification
    run: |
      curl -X POST ${ secrets.SLACK_WEBHOOK_URL } \
        -H 'Content-type: application/json' \
        --data '{"text": "🚨 XpressPro FX deployment FAILED on branch main. Check GitHub
Actions."}'
```

SECTION 5 — SECURITY HARDENING

9. Add helmet.js to Express app if not present:

```
import helmet from 'helmet'
app.use(helmet())
```

10. Confirm rate limiting on all auth/OTP/sensitive routes using express-rate-limit.

11. Confirm CORS allowlist uses ALLOWED_ORIGINS env var in production.

Never use origin: '*' in production.

12. Verify all user inputs use Zod validation schemas.

13. Confirm CSRF protection is active on all mutating endpoints (POST/PUT/DELETE).

14. Verify webhook signature validation for MoonPay and Coinbase integrations.

15. Ensure zero hardcoded secrets in any source file. Use dotenv in development only.

=====

SECTION 6 — PERFORMANCE & RELIABILITY

=====

16. Add compression middleware:

```
import compression from 'compression'
app.use(compression())
```

17. Add database indexes on all frequently queried columns in Prisma schema.

18. Ensure all list API endpoints are paginated.

19. Wrap ALL async route handlers with try/catch or a centralized asyncHandler wrapper.

20. Add a global Express error handler as the LAST middleware in app.ts:

```
app.use((err, req, res, next) => {
  logger.error(err)
  res.status(err.status || 500).json({
    error: NODE_ENV === 'production' ? 'Internal Server Error' : err.message
  })
})
```

=====

SECTION 7 — CODE QUALITY

=====

21. Fix ALL TypeScript errors. Zero errors on: `npx tsc --noEmit`

22. Fix ALL ESLint errors and warnings.

23. Replace ALL `console.log` calls with structured pino logger calls.

24. Remove all unused imports, variables, and dead code.

=====

SECTION 8 — ENVIRONMENT & DOCUMENTATION

=====

25. Create or update .env.example with every required variable:

DATABASE_URL=
SESSION_SECRET=
JWT_SECRET=
PORT=8080
NODE_ENV=production
ALLOWED_ORIGINS=https://yourdomain.com
MOONPAY_API_KEY=
COINBASE_WEBHOOK_SECRET=
SMTP_HOST=
SMTP_PORT=
SMTP_USER=
SMTP_PASS=

26. Required GitHub Secrets to add in GitHub → Settings → Secrets:

VPS_HOST, VPS_USER, VPS_SSH_KEY, VPS_PORT,
DOMAIN, SLACK_WEBHOOK_URL

=====

DEFINITION OF DONE — DO NOT STOP UNTIL ALL PASS

=====

- ✓ npx tsc --noEmit → zero errors
- ✓ npm run lint → zero errors
- ✓ npm audit --audit-level=high → zero high/critical vulnerabilities
- ✓ npm test → all tests pass
- ✓ npm run build (all workspaces) → completes with no errors
- ✓ node dist/index.js → starts without crashing
- ✓ GET /healthz → returns { "status": "ok" }
- ✓ railway.json → valid and correct
- ✓ ecosystem.config.js → valid PM2 cluster config
- ✓ nginx.conf → passes nginx -t validation
- ✓ deploy.sh → executable and tested
- ✓ deploy.yml → all jobs pass end-to-end
- ✓ .env.example → all required vars documented

Work through each section in order. For every fix, state what was wrong and what you changed. Do not skip any item. Prioritize in this exact order: Startup Crashes → Security → Reliability → Testing → Performance → Code Quality → Deployment Config.